

Contents

Contents	1
Topics to learn.....	6
1. Gradle project structure	6
2. Plain Java → Spring → Spring Boot	6
3. Spring Environment.....	6
4. Spring Bean Lifecycle (VERY IMPORTANT)	6
5. Configuration styles (CORE SKILL)	7
6. Dependency Injection patterns (DESIGN LEVEL)	7
7. Spring Boot auto-configuration (THE MAGIC)	7
8. Profiles + ConfigurationProperties (REAL WORLD)	8
9. Logging & Observability (ESSENTIAL)	8
10. Exception handling & Validation	8
11. Web layer (after foundations).....	9
12. Data access (JPA/JDBC)	9
13. Testing (VERY IMPORTANT).....	9
***** Plain Java *****	10
College class (simple POJO).....	10
Main class (YOU manage everything)	10
No Spring means:	11
Coupling	11
Factory-style Java	12
Why plain Java does NOT scale well	12
Interview	12
***** Spring framework *****	14
Example.....	14
package	14
Import Statements	14
Class declaration	15
main method.....	15
Creating Spring Container	15
Getting a bean from Spring.....	16
Calling business logic.....	16
Execution flow	16
***** Spring Boot *****	18

Your CURRENT code (Plain Spring).....	18
Above application using Spring Boot	18
What changed?	19
What @SpringBootApplication really doing?	19
BENEFITS of Spring Boot	20
How to EXECUTE Spring Boot app.....	21
Project structure (Spring Boot way).....	21
Interview	21
When should YOU use Spring Boot?	21
***** Environment in Spring *****	22
What Environment actually is?	22
Why Spring needs Environment	22
What can you do with Environment?.....	23
1. Read configuration properties	23
2. Check which profile is active	23
3. Provide default values safely.....	23
Why not just use @Value?	24
✓ Database switching	24
✓ Feature flags.....	24
✓ Logging config	24
How Spring populates Environment.....	25
Spring Boot vs Spring Framework	25
When SHOULD you use Environment?.....	25
When you should NOT	25
High level comparison	26
Hand on – Exercise to do	27
***** Create Project structure *****	27
Gradle build file	27
application.properties (BASE configuration).....	29
application-dev.yml (DEV profile)	29
application-prod.yml (PROD profile).....	29
Main application class	29
Environment reader bean (MOST IMPORTANT PART).....	30
Run scenarios (THIS IS WHERE YOU LEARN)	30
Property resolution order (MEMORIZE THIS)	32
***** Spring Boot 4.0.1 *****	33

build.gradle (Spring Boot + JDBC)	33
Plugins section	34
Java 25 Toolchain	34
Spring JDBC (not JPA)	34
Database Driver (runtime only)	34
Test configuration	34
How to run the application	35
***** REST API – Application *****	36
Create a Spring Boot project	36
Required dependency	36
Project structure (IMPORTANT)	36
Main Application class	37
Business logic (same as before)	37
REST Controller (NEW)	38
What annotations mean (VERY IMPORTANT).....	38
How execution works (FLOW).....	38
How to RUN the application	40
How to TEST the REST API.....	40
Why REST API is better than <code>main()</code> call.....	40
Interview one-liner.....	40
What next.....	41
***** DB Access *****	42
Layers in accessing a Database	42
Handle DB connection.....	42
Package Structure	42
Test / Execute.....	45
CommandLineRunner	45
***** Apache Log4j *****	46
XML Header.....	47
Configuration Root.....	47
Appenders (WHERE logs go)	47
Console Appender	47
Rolling File Appender	48
Loggers (WHAT logs to capture)	48
Spring Framework Logger	48
Application Logger	49

Root Logger	49
Log Flow Summary	49
Why this config is GOOD for Spring Boot.....	50
***** Login functionality – V1 *****	51
High-level Architecture (IMPORTANT)	51
Login Flow (What actually happens)	51
Oracle DB Table (users)	51
Dependencies.....	52
application.properties.....	52
User Entity (Plain Java)	52
☑ What you have achieved	54
***** Login functionality – Best practice *****	55
The BEST PRACTICE (Industry Standard)	55
The Recommended Architecture	55
JWT Token generation (MANDATORY)	56
Spring Security Filter (MANDATORY)	56
Role-Based Authorization (IMPORTANT)	57
Refresh Tokens (ADVANCED, optional initially)	57
Password Reset (SEPARATE feature)	57
*** Login system – THE BEST WAY ***	58
Steps.....	58
HandsOn.....	58
Database (Oracle).....	59
Dependencies (Gradle).....	59
application.properties.....	59
User Model.....	59
JDBC Repository	60
UserDetailsService (Spring Security Bridge).....	61
JWT Utility	62
JWT Authentication Filter	63
Security Configuration (MOST IMPORTANT)	64
Login Request / Response (Java records).....	64
Auth Controller (LOGIN API)	65
Protected REST API Example	66
How to Test (Postman)	66
MEMORIZE THIS	66

***** Front end – Spring Boot (REST API) *****	68
Where do HTML pages live in Spring Boot?	68
Project Structure (Typical)	68
Simple login.html (Frontend).....	69
REST Login API (Spring Boot).....	70
Login Controller.....	70
LoginRequest DTO	70
Service Layer (Oracle DB Check)	71
Oracle USERS Table (Example)	71
Other logic.....	71
When to use templates/ instead of static/?	71
**** Handling session id ****	72
How Session Works in Spring (Under the Hood)	72
Where Is the Session ID Stored?	72
How Is Session ID Sent to Other REST APIs?	72
Example: Frontend Fetch Call (Correct Way)	73
Pre-condition (Already done in your case)	73
HTML + JS REST Call (MOST IMPORTANT).....	73
Cross-Origin Case (Frontend & Backend on different ports/domains)	74
What NOT to Do.....	74
Backend: How Spring Reads the Session	74

Topics to learn

1. Gradle project structure

- ✓ `src/main/java`, `resources`, `build.gradle`
 - ✓ Tasks, dependencies, lifecycle
- This is **build-time understanding**

2. Plain Java → Spring → Spring Boot

- ✓ Object creation (`new`)
- ✓ IoC / DI
- ✓ `@Component`, `@Configuration`
- ✓ Auto-configuration

This is **runtime architecture understanding**

3. Spring Environment

- ✓ `application.properties` / `yaml`
- ✓ Profiles
- ✓ Property resolution order

This is **externalized configuration understanding**

4. Spring Bean Lifecycle (VERY IMPORTANT)

When exactly is my object created, initialized, destroyed?

As part of this, you will learn

- Bean creation phases
- `@PostConstruct`
- `@PreDestroy`
- `InitializingBean`
- `BeanPostProcessor`
- Lazy vs eager beans

Because:

- Environment is read **before beans**
- Auto-configuration depends on lifecycle

5. Configuration styles (CORE SKILL)

Understand **3 ways of configuring Spring**:

1. Java Config (@Configuration)
2. Component scanning
3. Auto-configuration (Boot magic)

Focus on:

```
@ConditionalOnProperty  
@ConditionalOnClass  
@Profile
```

This connects **Environment** → **Bean creation**

6. Dependency Injection patterns (DESIGN LEVEL)

Learn **how beans should depend on each other**:

- ✓ Constructor injection (best practice)
- ✗ Field injection (why it's bad)
- ✓ Qualifiers
- ✓ Primary beans

This is where **clean architecture** starts.

7. Spring Boot auto-configuration (THE MAGIC)

This is the **most misunderstood topic**.

Learn:

- `spring.factories`
- `AutoConfiguration.imports`
- How Boot decides **what to create**
- Why adding a dependency changes behavior

This explains *why adding `spring-boot-starter-data-jpa` suddenly creates a `DataSource`*

8. Profiles + ConfigurationProperties (REAL WORLD)

You already touched `Environment`, now master:

- `@ConfigurationProperties`
- `@Validated`
- Profile-specific configs:
 - `application-dev.yml`
 - `application-prod.yml`
 -

This is **enterprise-grade configuration**.

9. Logging & Observability (ESSENTIAL)

Learn:

- SLF4J
- Logback
- Log levels per profile
- MDC
- Structured logging

Why?

Logs are your **first debugger in production**

10. Exception handling & Validation

Learn:

- `@ControllerAdvice`
- Global exception handling
- Bean validation (`@NotNull`, `@Size`)
- Custom exceptions

This separates **toy apps** from **production apps**.

11. Web layer (after foundations)

Only now move to:

- Spring MVC
- REST controllers
- Filters vs Interceptors
- Request lifecycle

Why later?

Web is just **one entry point** to Spring.

12. Data access (JPA/JDBC)

Learn:

- Transaction boundaries
- `@Transactional`
- Entity lifecycle
- Lazy vs eager fetching

13. Testing (VERY IMPORTANT)

Learn:

- Unit vs integration tests
- `@SpringBootTest`
- Slice tests
- Mocking beans

***** Plain Java *****

College class (simple POJO)

```
package com.testing.qe;

public class College {

    public void principalName() {
        System.out.println("Principal is Dr. Sharma");
    }
}
```

If you look at above Java code

- No annotations
- No framework
- Just a normal Java class

Main class (YOU manage everything)

```
package com.testing.qe;

public class TestApplication {

    public static void main(String[] args) {

        // YOU create the object
        College college = new College();

        // YOU call the method
        college.principalName();
    }
}
```

No Spring means:

Responsibility	Who does it now?
Object creation	YOU
Dependency management	YOU
Lifecycle control	YOU
Configuration	YOU
Scalability support	×
Loose coupling	×

Coupling

With out Spring : Tight coupling

```
public class College {  
    private Principal principal = new Principal(); // hard-coded  
    public void showPrincipal() {  
        principal.name();  
    }  
}
```

Problems:

- Cannot replace `Principal`
- Hard to test
- Code changes needed everywhere

With Spring : Loose coupling

```
@Component  
public class College {  
    @Autowired  
    private Principal principal;  
}
```

Spring:

- Creates `Principal`
- Injects it
- Allows easy replacement

Factory-style Java

Before Spring existed, people did this

```
public class ObjectFactory {  
  
    public static College getCollege() {  
        return new College();  
    }  
}
```

Usage:

```
College college = ObjectFactory.getCollege();
```

Spring is basically a **VERY powerful, configurable factory**

Why plain Java does NOT scale well

Imagine:

- 50 classes
- Each depends on 5 others

Without Spring:

- 250+ *new* statements
- Tight coupling
- Nightmare to maintain

With Spring:

- Zero *new*
- Configuration-based wiring

Interview

- ✓ Small utility programs
- ✓ Simple learning demos
- ✓ One-class applications

- ✗ Enterprise apps
- ✗ REST APIs
- ✗ Database-heavy systems

***** Spring framework *****

Example

```
package com.testing.qe;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Test {
    public static void main(String[] args) {
        ApplicationContext context = new AnnotationConfigApplicationContext(CollegeConfig.class);
        College college = context.getBean("college", College.class);
        college.principalName();
    }
}
```

package

package com.testing.qe;

- This defines the **package (namespace)** of the class.
- It helps:
 - Organize code
 - Avoid class name conflicts
 - Enable **component scanning** in Spring

Import Statements

import org.springframework.context.ApplicationContext;

import org.springframework.context.annotation.AnnotationConfigApplicationContext;

ApplicationContext	AnnotationConfigApplicationContext
• Central Spring IoC container • Responsible for: ○ Creating objects (beans) ○ Managing their lifecycle ○ Injecting dependencies Think of it as Spring's factory + manager.	• A concrete implementation of <code>ApplicationContext</code> • Used when: ○ Configuration is done using annotations ○ No <code>applicationContext.xml</code> is used This class: • Reads <code>@Configuration</code> • Scans <code>@Component</code> • Handles <code>@Bean</code>, <code>@Autowired</code>, etc.

Class declaration

```
public class Test {
```

This is a **plain Java class**, not managed by Spring.

⚠️ Important:

- `Test` itself is **not a Spring bean**
- It just **boots the Spring container**

main method

```
public static void main(String[] args) {
```

- Java application entry point
- Starts Spring manually (not Spring Boot)

Creating Spring Container

```
ApplicationContext context = new AnnotationConfigApplicationContext(CollegeConfig.class);
```

What happens here step-by-step:

1. Spring container is created
2. `CollegeConfig.class` is loaded

```
@Configuration

@ComponentScan("com.testing.qe")

public class CollegeConfig {

}
```

3. Spring:

- Reads `@Configuration`
- Scans packages for:
 - `@Component`
 - `@Service`
 - `@Repository`
 - `@Controller`
- Creates beans

At this moment, all beans are created and stored in the container.

Getting a bean from Spring

```
College college = context.getBean("college", College.class);
```

What this means:

- "college" → **bean ID**
- `College.class` → expected type

Spring:

1. Looks for a bean named **college**
2. Confirms it is of type `College`
3. Returns the object

Usually defined as:

```
@Component
public class College {
    public void principalName() {
        System.out.println("Principal is Dr. Sharma");
    }
}
```

Bean name defaults to **class name with first letter lowercase**.

Calling business logic

```
college.principalName();
```

- Calls method on Spring-managed object
- Any dependencies inside `College` are:
 - Already injected
 - Already initialized

Execution flow

JVM starts

↓

main() method executes

↓

Spring container starts

↓

Configuration class loaded

↓

Beans created & wired

↓

getBean() fetches object



Method executed

***** Spring Boot *****

Compare **your above Spring code** with **Spring Boot**, then we will cover **benefits** and **how to execute**.

Your CURRENT code (Plain Spring)

```
ApplicationContext context = new AnnotationConfigApplicationContext(CollegeConfig.class);
```

```
College college = context.getBean("college", College.class);
```

```
college.principalName();
```

What YOU do manually

- Create `ApplicationContext`
- Create `@Configuration` class
- Add `@ComponentScan`
- Manage dependencies
- Decide how the app starts

This is **pure Spring Framework**.

Above application using Spring Boot

Main Class

```
package com.testing.qe;
```

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
```

```
@SpringBootApplication
public class TestApplication {

    public static void main(String[] args) {
        ApplicationContext context = SpringApplication.run(TestApplication.class, args);
        College college = context.getBean(College.class);
        college.principalName();
    }
}
```

No `AnnotationConfigApplicationContext`

No `CollegeConfig` needed

College class (almost same)

```
package com.testing.qe;

import org.springframework.stereotype.Component;

@Component
public class College {
    public void principalName() {
        System.out.println("Principal is Dr. Sharma");
    }
}
```

What changed?

Plain Spring	Spring Boot
Manual config class	✗ Not needed
Manual component scan	✗ Auto
XML / Java config	✗ Optional
Embedded server	✗
Boilerplate code	✗

What @SpringBootApplication really doing?

@SpringBootApplication

This is **3 annotations combined**

@Configuration

@EnableAutoConfiguration

@ComponentScan

Meaning:

- @Configuration → Java config
- @ComponentScan → scans current package & subpackages
- @EnableAutoConfiguration → magic ✨ (auto setup)

BENEFITS of Spring Boot

✓ 1. No configuration headache

- No XML
- No manual bean wiring
- No server setup

✓ 2. Auto-configuration (HUGE)

Example:

- Add `spring-boot-starter-web`
- BOOM ☐
 - Tomcat configured
 - DispatcherServlet created
 - JSON mapper ready

You didn't write **anything**.

✓ 3. Embedded server

No need to install:

- Tomcat
- JBoss
- WebLogic

Spring Boot ships with **embedded Tomcat**.

✓ 4. Production-ready

Spring Boot gives:

- Health checks
- Metrics
- Logging
- Externalized config

How to EXECUTE Spring Boot app

Option 1: From IDE (IntelliJ / Eclipse)

- Right-click on `TestApplication`
- Click **Run**

✓ Most common

Option 2: Maven command

```
mvn spring-boot:run
```

Option 3: Run JAR (production way)

```
mvn clean package java -jar target/test-0.0.1-SNAPSHOT.jar
```

Project structure (Spring Boot way)

```
src/main/java
├── com.testing.qe
│   ├── TestApplication.java  <-- main class
│   └── College.java
```

Main class must be at top package level

(or component scanning will fail)

Interview

Spring Boot = Spring + Auto Configuration + Embedded Server + Zero Boilerplate

When should YOU use Spring Boot?

- ✓ REST APIs
- ✓ Microservices
- ✓ Real-world projects
- ✓ Faster development

***** Environment in Spring *****

`org.springframework.core.env.Environment` is **one of the most important abstractions in Spring**, but it's often used without really understanding *why*.

What Environment actually is?

In **Spring Framework**, `Environment` is an **interface** that represents:

The current runtime environment of your application

It gives you **read-only access** to:

- Configuration **properties**
- Active **profiles**
- Default profiles

Why Spring needs `Environment`

Modern applications don't hardcode values like:

- Database URL
- Username/password
- Port numbers
- Feature flags

Instead, these values change by **environment**:

- dev
- test
- qa
- prod

`Environment` lets Spring **centralize and resolve** all of this configuration.

What can you do with `Environment`?

1. Read configuration properties

```
@Autowired
private Environment environment;

String url = environment.getProperty("spring.datasource.url");
```

This value could come from:

- `application.properties`
- `application.yml`
- OS environment variables
- JVM system properties
- Command-line arguments

You **don't care where it came from** — Spring resolves it.

2. Check which profile is active

```
if (environment.acceptsProfiles("prod")) {
    // production-only logic
}
```

Or:

```
String[] profiles = environment.getActiveProfiles();
```

This is extremely useful for:

- Logging
- Conditional behavior
- Feature toggles

3. Provide default values safely

```
String timeout =
    environment.getProperty("app.timeout", "30");
```

If `app.timeout` is missing → "30" is used.

No `NullPointerException`.

Why not just use `@Value`?

You *can* do this:

```
@Value("${server.port}")
private int port;
```

But `Environment` is better when:

Use case	Why <code>Environment</code> is better
Dynamic access	Can read properties anytime
Conditional logic	Profiles + properties together
Optional properties	Built-in defaults
Libraries/frameworks	Cannot rely on <code>@Value</code>

Typical real-world use cases

✓ Database switching

```
if (environment.acceptsProfiles("dev")) { // H2 database } else { // Oracle /
MySQL }
```

✓ Feature flags

```
if (environment.getProperty("feature.newUI", Boolean.class, false)) {
enableNewUI(); }
```

✓ Logging config

```
log.info("Active profile: {}", Arrays.toString(environment.getActiveProfiles()));
```

How Spring populates `Environment`

Spring collects properties from **PropertySources**, in this order:

1. Command-line args
2. JVM system properties
3. OS environment variables
4. `application.properties` / `yaml`
5. `@PropertySource`

You can inspect them:

```
ConfigurableEnvironment env = (ConfigurableEnvironment) environment;  
env.getPropertySources().forEach(ps -> System.out.println(ps.getName()));
```

Spring Boot vs Spring Framework

In **Spring Boot**:

- Environment is **auto-configured**
- `application.properties` is loaded automatically
- Profiles are first-class citizens

In plain Spring:

- You must define property sources manually

But the **Environment API is identical**.

When SHOULD you use `Environment`?

- ✓ Framework / infrastructure code
- ✓ Conditional configuration
- ✓ Reading optional properties
- ✓ Profile-based behaviour

When you should NOT

- ✗ Simple POJOs
- ✗ Business logic classes
- ✗ Where `@ConfigurationProperties` fits better

High level comparision

Aspect	Environment	@Value	@ConfigurationProperties 
Purpose	Programmatic access to config	Inject single value	Bind structured config
Style	Imperative (code-driven)	Declarative (annotation)	Declarative + type-safe
Type safety	✗ No	⚠ Partial	✓ Yes
Supports defaults	✓ Yes	⚠ Limited	✓ Yes
Best for	Framework / infra logic	Simple values	Application config
Testability	⚠ Medium	✗ Poor	✓ Excellent
Validation	✗ No	✗ No	✓ Yes (<code>@Validated</code>)

Hand on – Exercise to do

***** Create Project structure *****

```
demo-env/  
├─ build.gradle  
└─ src/  
    └─ main/  
        ├─ java/  
        │   └─ com/example/envdemo/  
        │       ├─ EnvDemoApplication.java  
        │       └─ EnvPrinter.java  
        └─ resources/  
            ├─ application.properties  
            ├─ application-dev.yml  
            └─ application-prod.yml
```

Gradle build file

```
plugins {  
    id 'java'  
    id 'org.springframework.boot' version '3.2.1'  
    id 'io.spring.dependency-management' version '1.1.4'  
}  
  
group = 'com.example'  
version = '0.0.1-SNAPSHOT'  
  
java {  
    toolchain {  
        languageVersion = JavaLanguageVersion.of(17)  
    }  
}  
  
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter'  
}
```

Note: This uses **Spring Boot**, which auto-wires the `Environment`

application.properties (BASE configuration)

```
app.name=Env Demo App
app.timeout=30
app.source=application.properties
```

application-dev.yml (DEV profile)

```
app:
  timeout: 10
  source: application-dev.yml
```

Note: Overrides only when **dev** profile is active.

application-prod.yml (PROD profile)

```
app:
  timeout: 60
  source: application-prod.yml
```

Note: Overrides only when **prod** profile is active.

Main application class

```
package com.example.envdemo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class EnvDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(EnvDemoApplication.class, args);
    }
}
```

Environment reader bean (MOST IMPORTANT PART)

```
package com.example.envdemo;

import jakarta.annotation.PostConstruct;
import org.springframework.core.env.Environment;
import org.springframework.stereotype.Component;

import java.util.Arrays;

@Component
public class EnvPrinter {

    private final Environment environment;

    public EnvPrinter(Environment environment) {
        this.environment = environment;
    }

    @PostConstruct
    public void print() {
        System.out.println("---- ENVIRONMENT DEMO ----");

        System.out.println("Active profiles: "
            + Arrays.toString(environment.getActiveProfiles()));

        System.out.println("app.name = "
            + environment.getProperty("app.name"));

        System.out.println("app.timeout = "
            + environment.getProperty("app.timeout"));

        System.out.println("app.source = "
            + environment.getProperty("app.source"));

        System.out.println("-----");
    }
}
```

Note: This bean runs **after the Environment is prepared**, but **before the app is fully ready**.

Run scenarios (THIS IS WHERE YOU LEARN)

Case 1: No profile (default)

bash

 Copy code

```
./gradlew bootRun
```

Output:

yaml

 Copy code

```
Active profiles: []
app.name = Env Demo App
app.timeout = 30
app.source = application.properties
```

✓ Only `application.properties` used

Case 2: DEV profile

bash

 Copy code

```
./gradlew bootRun --args='--spring.profiles.active=dev'
```

Output:

yaml

 Copy code

```
Active profiles: [dev]
app.name = Env Demo App
app.timeout = 10
app.source = application-dev.yml
```

✓ `application-dev.yml` overrides base

Case 3: PROD profile

bash

 Copy code

```
./gradlew bootRun --args='--spring.profiles.active=prod'
```

Output:

r

 Copy code

```
Active profiles: [prod]
app.name = Env Demo App
app.timeout = 60
app.source = application-prod.yml
```

Case 4: Command-line override (HIGHEST PRIORITY)

bash

 Copy code

```
./gradlew bootRun --args='--spring.profiles.active=dev --app.timeout=99'
```

Output:

yaml

 Copy code

```
Active profiles: [dev]
app.timeout = 99
```

✓ Command-line wins over everything

Property resolution order (MEMORIZE THIS)

From **highest** → **lowest** priority:

1. **Command-line arguments**
2. JVM system properties (`-D`)
3. OS environment variables
4. `application-{profile}.yml`
5. `application.properties`
6. Default values in code

That's why `--app.timeout=99` wins.

***** Spring Boot 4.0.1 *****

As of now, **Java 25** and **Spring Boot 4.0.1** are **future / preview versions**. The configuration below shows the **correct structure and intent**, but you may need to adjust versions if a specific plugin does not yet officially support Java 25.

- Spring Boot 4.x may not yet support Java 25 , Temporarily use **Java 21 or 23** until official support

build.gradle (Spring Boot + JDBC)

```
plugins {  
    id 'java'  
    id 'org.springframework.boot' version '4.0.1'  
    id 'io.spring.dependency-management' version '1.1.6'  
}  
  
group = 'com.example'  
version = '0.0.1-SNAPSHOT'  
  
java {  
    toolchain {  
        languageVersion = JavaLanguageVersion.of(25)  
    }  
}  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {  
  
    // Spring Boot Core  
    implementation 'org.springframework.boot:spring-boot-starter'  
  
    // Spring JDBC  
    implementation 'org.springframework.boot:spring-boot-starter-jdbc'  
  
    // Database Driver (example: Oracle)  
    runtimeOnly 'com.oracle.database.jdbc:ojdbc11'  
  
    // Testing  
    testImplementation 'org.springframework.boot:spring-boot-starter-test'  
}  
  
tasks.named('test') {  
    useJUnitPlatform()  
}
```

Plugins section

```
plugins {  
    id 'java'  
    id 'org.springframework.boot' version '4.0.1'  
    id 'io.spring.dependency-management' version '1.1.6'  
}
```

java → enables Java compilation

spring-boot → packaging, auto-configuration, executable JAR

dependency-management → lets Spring control dependency versions

Java 25 Toolchain

```
java {  
    toolchain {  
        languageVersion = JavaLanguageVersion.of(25)  
    }  
}
```

✓ Ensures:

Gradle uses JDK 25

Build is consistent across machines

No reliance on JAVA_HOME

Spring JDBC (not JPA)

implementation 'org.springframework.boot:spring-boot-starter-jdbc'

Includes:

- JdbcTemplate
 - DataSource
 - Transaction management
 - Connection pooling (HikariCP)
-
- Lightweight
 - Perfect for learning JDBC
 - Better control than JPA for beginners

Database Driver (runtime only)

runtimeOnly 'com.oracle.database.jdbc:ojdbc11'

Why `runtimeOnly`?

- Driver is **not needed at compile time**
- Only required when app runs

Test configuration

testImplementation 'org.springframework.boot:spring-boot-starter-test'

Includes:

- JUnit 5
- Mockito
- Spring Test utilities

How to run the application

Build

```
./gradlew clean build
```

Run

```
./gradlew bootRun
```

Run JAR

```
java -jar build/libs/*.jar
```

***** REST API – Application *****

Instead of calling:

```
college.principalName();
```

We will expose it as a **REST endpoint**:

GET <http://localhost:8080/college/principal>

and get a response like:

```
Principal is Dr. Sharma
```

Create a Spring Boot project

Use: IntelliJ / Eclipse **OR**

Required dependency

Add only **Spring Web**

(Maven dependency)

```
<dependency>
  <groupId>org.springframework.boot</groupId>

  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

This is powered by **Spring Boot**, built on **Spring Framework**.

Project structure (IMPORTANT)

```
src/main/java
├── com.testing.qe
│   ├── TestApplication.java ← main class
│   ├── College.java         ← business logic
│   └── CollegeController.java ← REST API
```

Main class must be at **top package level**.

Main Application class

```
package com.testing.qe;
```

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
public class TestApplication {

    public static void main(String[] args) {
        SpringApplication.run(TestApplication.class, args);
    }
}
```

✓ No `ApplicationContext`

✓ No config class

✓ No manual wiring

Business logic (same as before)

```
College.java
```

```
package com.testing.qe;
```

```
import org.springframework.stereotype.Component;
```

```
@Component
public class College {

    public String principalName() {
        return "Principal is Dr. Sharma";
    }
}
```

✓ Spring manages this bean

✓ No `new College()` anywhere

REST Controller (NEW)

CollegeController.java

```
package com.testing.qe;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
public class CollegeController {

    @Autowired
    private College college;

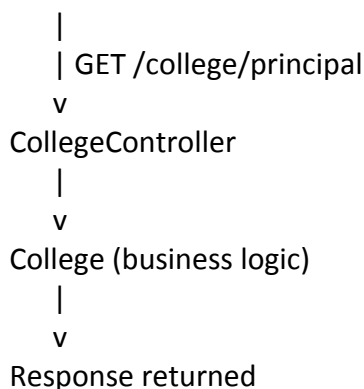
    @GetMapping("/college/principal")
    public String getPrincipalName() {
        return college.principalName();
    }
}
```

What annotations mean (VERY IMPORTANT)

Annotation	Purpose
@RestController	Marks class as REST API
@GetMapping	Maps HTTP GET request
@Autowired	Injects <code>College</code> bean
@Component	Makes class a Spring bean
@SpringBootApplication	Auto config + scan

How execution works (FLOW)

Browser / Postman



How to RUN the application

✓ Option 1: From IDE

- Right-click `TestApplication`
- Click **Run**

✓ Option 2: Maven

```
mvn spring-boot:run
```

✓ Option 3: JAR (production style)

```
mvn clean package java -jar target/*.jar
```

How to TEST the REST API

In browser

<http://localhost:8080/college/principal>

In curl

```
curl http://localhost:8080/college/principal
```

Why REST API is better than `main()` call

Old way

Console output

Local JVM

Not reusable

Hard to scale

REST way

HTTP response

Network accessible

Used by UI / Mobile

Cloud-ready

Interview one-liner

Spring Boot REST API exposes business logic using HTTP endpoints, making applications scalable, reusable, and platform-independent.

What next

- Convert response to **JSON**
- Add **POST API**
- Add **DTO**
- Connect to **Oracle DB**
- Explain **@RestController** vs **@Controller**

***** DB Access *****

Spring Boot manages the connection pool for your project

You should **never manually open/close connections everywhere.**

Layers in accessing a Database

Controller → Service → Repository (DAO) → Database

- **Controller** → handles HTTP requests (optional if not REST)
- **Service** → business logic
- **Repository (DAO)** → all SQL (SELECT / UPDATE / DELETE)
- **DataSource** → manages JDBC connections (Spring handles this)

Handle DB connection

Use **DataSource + JdbcTemplate**

Spring Boot auto-configures this from Properties file

```
spring.datasource.url=jdbc:oracle:thin:@localhost:1521/orclpdb
spring.datasource.username=app_user
spring.datasource.password=secret
spring.datasource.driver-class-name=oracle.jdbc.OracleDriver
```

Package Structure

This follows **Java + Spring standards.**

```
com.example.college
├── controller
│   └── StudentController.java
├── service
│   └── StudentService.java
├── repository
│   └── StudentRepository.java
├── model
│   └── Student.java
└── CollegeApplication.java
```

CollegeApplication

```

package com.example.college;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class CollegeApplication {

    public static void main(String[] args) {
        SpringApplication.run(CollegeApplication.class, args);
    }
}

```

Controller

optional, if REST API

```

@RestController
@RequestMapping("/students")
public class StudentController {

    private final StudentService service;

    public StudentController(StudentService service) {
        this.service = service;
    }

    @GetMapping
    public List<Student> getAll() {
        return service.getStudents();
    }
}

```

Model Class

- ✓ No DB code
- ✓ Just data

```

package com.example.college.model;

public class Student {

    private Long id;
    private String name;
    private Integer age;

    // getters and setters
}

```

Repository layer (ALL SQL goes here)

- One class = **one table**
- All SQL in **one place**
- Uses **prepared statements automatically**
- No connection handling code

Service layer (business logic)

- ✓ No SQL
- ✓ Easy to test
- ✓ Business rules go here

```

package com.example.college.repository;
import com.example.college.model.Student;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public class StudentRepository {
    private final JdbcTemplate jdbcTemplate;

    public StudentRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    // SELECT
    public List<Student> findAll() {
        String sql = "SELECT id, name, age FROM student";
        return jdbcTemplate.query(
            sql,
            (rs, rowNum) -> {
                Student s = new Student();
                s.setId(rs.getLong("id"));
                s.setName(rs.getString("name"));
                s.setAge(rs.getInt("age"));
                return s;
            }
        );
    }

    // INSERT
    public int save(Student student) {
        String sql = "INSERT INTO student (id, name, age) VALUES (?, ?, ?)";
        return jdbcTemplate.update(sql,
            student.getId(),
            student.getName(),
            student.getAge());
    }

    // UPDATE
    public int updateName(Long id, String name) {
        String sql = "UPDATE student SET name = ? WHERE id = ?";
        return jdbcTemplate.update(sql, name, id);
    }

    // DELETE
    public int deleteById(Long id) {
        String sql = "DELETE FROM student WHERE id = ?";
        return jdbcTemplate.update(sql, id);
    }
}

```

```

package com.example.college.service;
import com.example.college.model.Student;
import com.example.college.repository.StudentRepository;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class StudentService {
    private final StudentRepository repository;

    public StudentService(StudentRepository repository) {
        this.repository = repository;
    }

    public List<Student> getStudents() {
        return repository.findAll();
    }

    public void createStudent(Student student) {
        repository.save(student);
    }

    public void updateStudentName(Long id, String name) {
        repository.updateName(id, name);
    }

    public void deleteStudent(Long id) {
        repository.deleteById(id);
    }
}

```

Test / Execute

```
@Bean
CommandLineRunner test(StudentService service) {
    return args -> {
        Student s = new Student();
        s.setId(1L);
        s.setName("Ram");
        s.setAge(20);

        service.createStudent(s);
    };
}
```

CommandLineRunner

- Code sits Inside your Java app
- Runs **after Spring starts**
- Has access to DB, services, repositories

It is a **Spring Boot startup hook**, Meaning:

- When your **Spring Boot application starts**
- After Spring creates all beans (DataSource, Repository, Service)
- This code **automatically runs once**

./gradlew build

java -jar build/libs/college-0.0.1-SNAPSHOT.jar

This is how above code looks like when written in Manual Object creation and when used lambda

CommandLineRunner as a normal interface

```
public interface CommandLineRunner {
    void run(String[] args);
}
```

Manual object creation & method call

```
public class MainApp {

    public static void main(String[] args) {

        // 1. Manually create dependencies
        StudentService service = new StudentService();

        // 2. Manually create the runner
        CommandLineRunner runner = new CommandLineRunner() {
            @Override
            public void run(String[] args) {
                Student s = new Student();
                s.setId(1L);
                s.setName("Ram");
                s.setAge(20);

                service.createStudent(s);
            }
        };

        // 3. Manually execute it
        runner.run(args);
    }
}
```

Plain Java using lambda

```
public class MainApp {

    public static void main(String[] args) {

        StudentService service = new StudentService();

        CommandLineRunner runner = (appArgs) -> {
            Student s = new Student();
            s.setId(1L);
            s.setName("Ram");
            s.setAge(20);

            service.createStudent(s);
        };

        runner.run(args);
    }
}
```

***** Apache Log4j *****

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN">

    <Appenders>

        <!-- Console output -->
        <Console name="Console" target="SYSTEM_OUT">
            <PatternLayout>
                pattern="%d{yyyy-MM-dd HH:mm:ss} %-5level [%t] %c{1} - %msg%n"/>
            </Console>

        <!-- File logging -->
        <RollingFile name="FileLogger"
            fileName="logs/application.log"
            filePattern="logs/application-%d{yyyy-MM-dd}.log">
            <PatternLayout>
                <Pattern>%d{yyyy-MM-dd HH:mm:ss} %-5level %c{1} - %msg%n</Pattern>
            </PatternLayout>
            <Policies>
                <TimeBasedTriggeringPolicy/>
            </Policies>
        </RollingFile>

    </Appenders>

    <Loggers>

        <!-- Spring framework logs -->
        <Logger name="org.springframework" level="INFO"/>

        <!-- Your application logs -->
        <Logger name="com.shrivardhan.springboot" level="DEBUG" additivity="false">
            <AppenderRef ref="Console"/>
            <AppenderRef ref="FileLogger"/>
        </Logger>

        <Root level="INFO">
            <AppenderRef ref="Console"/>
        </Root>

    </Loggers>

</Configuration>
```

This is a **Log4j2 XML configuration file**. It tells Log4j **where to write logs, how to format them, and which logs to capture**. I'll explain it **section by section**, in simple terms, and also relate it to a **Spring Boot application**.

XML Header

```
<?xml version="1.0" encoding="UTF-8"?>
```

Standard XML declaration

UTF-8 ensures all characters are supported

Configuration Root

```
<Configuration status="WARN">
```

What it means

- This is the **root element** of Log4j2 configuration
- `status="WARN"` controls **Log4j's internal logging**
 - Only warnings or errors from Log4j itself will be printed
 - Helps avoid clutter during startup

Appenders (WHERE logs go)

```
<Appenders>
```

Appenders define **destinations** for logs (console, file, database, etc.)

Console Appender

```
<Console name="Console" target="SYSTEM_OUT">
```

Purpose: Prints logs to the **terminal / console**

```
<PatternLayout
```

```
    pattern="%d{yyyy-MM-dd HH:mm:ss} %-5level [%t] %c{1} - %msg%n"/>
```

Pattern Meaning

<code>%d{yyyy-MM-dd HH:mm:ss}</code>	Date & time
<code>%-5level</code>	Log level (INFO, DEBUG, ERROR)
<code>[%t]</code>	Thread name
<code>%c{1}</code>	Logger class name (short form)
<code>%msg</code>	Log message
<code>%n</code>	New line

Example:

2026-01-25 10:45:30 INFO [main] UserService - User created

Rolling File Appender

```
<RollingFile name="FileLogger"
    fileName="logs/application.log"
    filePattern="logs/application-%d{yyyy-MM-dd}.log">
```

Purpose

- Writes logs to a **file**
- Automatically **creates a new file every day**

File Purpose

application.log Current log file

application-2026-01-25.log Archived daily log

```
<PatternLayout>
    <Pattern>%d{yyyy-MM-dd HH:mm:ss} %-5level %c{1} - %msg%n</Pattern>
</PatternLayout>
```

Similar to console but **without thread name** (simpler for file logs)

```
<Policies>
    <TimeBasedTriggeringPolicy/>
</Policies>
```

Create a new log file every day

Loggers (WHAT logs to capture)

Loggers control **which packages/classes are logged and at what level**

Spring Framework Logger

```
<Logger name="org.springframework" level="INFO"/>
```

Meaning

- Logs from Spring framework:
 - INFO, WARN, ERROR
 - **✗** DEBUG logs are ignored

✓ Keeps Spring logs **clean and minimal**

Application Logger

```
<Logger name="com.shrivardhan.springboot" level="DEBUG" additivity="false">
```

Meaning

- Applies to **your application package**
- DEBUG level → logs **everything**
- `additivity="false"` prevents logs from being duplicated

Appenders used

```
<AppenderRef ref="Console"/>  
<AppenderRef ref="FileLogger"/>
```

➡ Logs go to:

- Console
- File

Root Logger

```
<Root level="INFO">  
    <AppenderRef ref="Console"/>  
</Root>
```

Meaning

- Default logger for **all other packages**
- INFO and above only
- Logs only to **console**

Log Flow Summary

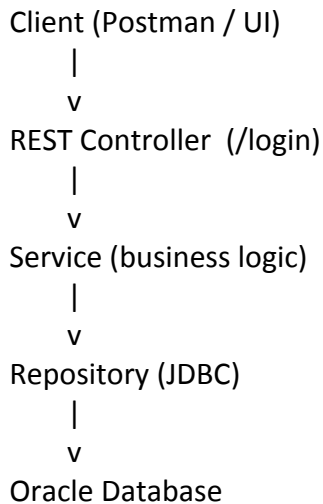
Source	Level	Console	File
Spring framework	INFO+	✓	✗
<code>com.shrivardhan.springboot</code>	DEBUG+	✓	✓
Other libraries	INFO+	✓	✗

Why this config is GOOD for Spring Boot

- ✓ Separates **framework logs** from **application logs**
- ✓ Daily rolling files (safe for production)
- ✓ DEBUG enabled only for your code
- ✓ Prevents log duplication
- ✓ Clean console output

***** Login functionality – V1 *****

High-level Architecture (IMPORTANT)



Login Flow (What actually happens)

1. Client sends **username + password**
2. API fetches user by username from Oracle DB
3. Stored **hashed password** is compared
4. If valid → return **success / token**
5. If invalid → return **401 Unauthorized**

Oracle DB Table (users)

```
CREATE TABLE USERS (  
    USER_ID NUMBER PRIMARY KEY,  
    USERNAME VARCHAR2(50) UNIQUE NOT NULL,  
    PASSWORD VARCHAR2(255) NOT NULL,  
    ENABLED CHAR(1) DEFAULT 'Y'  
);
```

⚠️ **Never store plain passwords**

Store **BCrypt** hashed passwords.

Dependencies

```
implementation 'org.springframework.boot:spring-boot-starter-web'
implementation 'org.springframework.boot:spring-boot-starter-jdbc'
implementation 'org.springframework.security:spring-security-crypto'
runtimeOnly 'com.oracle.database.jdbc:ojdbc11'
```

spring-security-crypto is only for **password hashing**, not full security yet.

application.properties

```
spring.datasource.url=jdbc:oracle:thin:@localhost:1521/orclpdb
spring.datasource.username=app_user
spring.datasource.password=app_password
spring.datasource.driver-class-name=oracle.jdbc.OracleDriver

spring.sql.init.mode=never
```

User Entity (Plain Java)

User Entity (Plain Java)	Service (Login Logic)
<pre>public class User { private Long userId; private String username; private String password; private boolean enabled; // getters and setters }</pre>	<pre>@Service public class AuthService { private final UserRepository userRepository; private final PasswordEncoder passwordEncoder; public AuthService(UserRepository userRepository) { this.userRepository = userRepository; this.passwordEncoder = new BCryptPasswordEncoder(); } public boolean login(String username, String rawPassword) { User user = userRepository.findByUsername(username) .orElseThrow(() -> new RuntimeException("Invalid credentials")); if (!user.isEnabled()) { throw new RuntimeException("User disabled"); } return passwordEncoder.matches(rawPassword, user.getPassword()); } }</pre>

Repository (JDBC Layer)

```
@Repository
public class UserRepository {

    private final JdbcTemplate jdbcTemplate;

    public UserRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public Optional<User> findByUsername(String username) {
        String sql = """
            SELECT USER_ID, USERNAME, PASSWORD, ENABLED
            FROM USERS
            WHERE USERNAME = ?
            """;

        return jdbcTemplate.query(sql, rs -> {
            if (rs.next()) {
                User user = new User();
                user.setUserId(rs.getLong("USER_ID"));
                user.setUsername(rs.getString("USERNAME"));
                user.setPassword(rs.getString("PASSWORD"));
                user.setEnabled("Y".equals(rs.getString("ENABLED")));
                return Optional.of(user);
            }
            return Optional.empty();
        }, username);
    }
}
```

Rest Controller

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequest request) {

        boolean success = authService.login(
            request.username(),
            request.password()
        );

        if (success) {
            return ResponseEntity.ok("Login successful");
        }
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
            .body("Invalid username or password");
    }
}
```

Login Request DTO <pre> public record LoginRequest(String username, String password) {} </pre> ✓ Clean ✓ Immutable ✓ Perfect for REST APIs	Password Hashing (IMPORTANT) Create password (one-time utility) <pre> public static void main(String[] args) { PasswordEncoder encoder = new BCryptPasswordEncoder(); System.out.println(encoder.encode("myPassword123")); } </pre> Store output in DB
Test with Postman POST <pre> http://localhost:8080/api/auth/login </pre> Body (JSON) <pre> { "username": "admin", "password": "myPassword123" } </pre>	

🔗 What you have achieved

- ✓ REST login API
- ✓ Oracle DB authentication
- ✓ JDBC (no JPA magic)
- ✓ Secure password hashing
- ✓ Latest Java features (records)
- ✓ Clean layered design

***** Login functionality – Best practice *****

In a **REST API**:

- The server **does NOT remember users**
- Every request must prove **who the user is**
- This is called **stateless authentication**

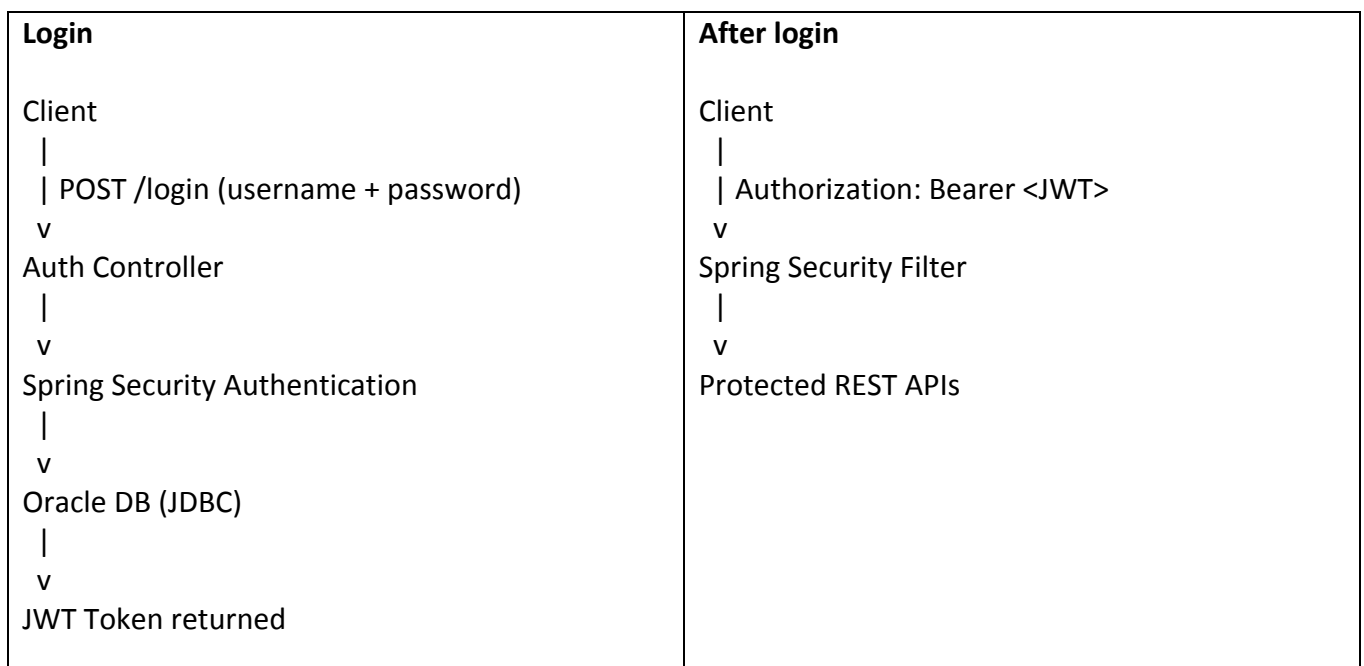
The BEST PRACTICE (Industry Standard)

Use token-based authentication with JWT + Spring Security

This is the **most widely accepted approach** for:

- REST APIs
- Microservices
- Mobile apps
- Frontend (React / Angular)
- Cloud deployments

The Recommended Architecture



JWT Token generation (MANDATORY)

What it is

A **JWT** is a signed token that proves:

- who the user is
- what roles they have
- that they already logged in

Why it exists

- ✓ Avoid sessions
- ✓ Stateless
- ✓ Scales easily
- ✓ Secure

Example

Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

Spring Security Filter (MANDATORY)

Implemented using Spring Security

What it does

- Runs **before every request**
- Extracts JWT from header
- Validates token
- Sets authenticated user in context

Your controller never manually checks login.

Why it exists

- ✓ Centralized security
- ✓ No duplicate code
- ✓ Battle-tested

Role-Based Authorization (IMPORTANT)

What it is

Controls who can access what

```
@PreAuthorize("hasRole('ADMIN')")  
@GetMapping("/admin")
```

Why it exists

- ✓ Principle of least privilege
- ✓ Enterprise requirement

Refresh Tokens (ADVANCED, optional initially)

Problem

JWTs should be **short-lived** (15–30 mins)

Solution

- Access Token → short life
- Refresh Token → long life (stored in DB)

□ Used by **frontend apps**

Do you need it now?

✗ NO (add later)

Password Reset (SEPARATE feature)

This is **NOT** part of login

It's a **user-management** feature:

- Forgot password
- Email link
- OTP
- Reset flow

✗ Don't mix this with authentication logic

*** Login system – THE BEST WAY ***

The best way to implement REST login is:

Spring Security + JDBC + BCrypt + JWT + stateless authentication.

Steps

Modern REST login, Implement this (in this order):

✓ Step 1: Spring Security + JDBC authentication

- UserDetailsService
- PasswordEncoder (BCrypt)
- Oracle DB

✓ Step 2: JWT-based login

- /login returns JWT
- Client stores token

✓ Step 3: JWT filter

- Validate token for every request

✓ Step 4: Role-based access

- @PreAuthorize
- @RolesAllowed

HandsOn

A complete, clean, production-grade Spring Security + JWT login implementation using:

- ✓ REST API
- ✓ Spring Boot
- ✓ Spring Security
- ✓ JDBC (Oracle)
- ✓ BCrypt password hashing
- ✓ Stateless JWT authentication
- ✓ Latest Java style (records, constructor injection)

Database (Oracle)

```
CREATE TABLE USERS (  
    USER_ID NUMBER PRIMARY KEY,  
    USERNAME VARCHAR2(50) UNIQUE NOT NULL,  
    PASSWORD VARCHAR2(255) NOT NULL,  
    ROLE VARCHAR2(30) NOT NULL,  
    ENABLED CHAR(1) DEFAULT 'Y'  
);
```

Password must be **BCrypt hashed**

Dependencies (Gradle)

```
implementation 'org.springframework.boot:spring-boot-starter-web'  
implementation 'org.springframework.boot:spring-boot-starter-security'  
implementation 'org.springframework.boot:spring-boot-starter-jdbc'  
implementation 'io.jsonwebtoken:jjwt-api:0.11.5'  
runtimeOnly 'io.jsonwebtoken:jjwt-impl:0.11.5'  
runtimeOnly 'io.jsonwebtoken:jjwt-jackson:0.11.5'  
implementation 'org.springframework.security:spring-security-crypto'  
runtimeOnly 'com.oracle.database.jdbc:ojdbc11'
```

application.properties

```
spring.datasource.url=jdbc:oracle:thin:@localhost:1521/orclpdb  
spring.datasource.username=app_user  
spring.datasource.password=app_password  
  
jwt.secret=MyVerySecretJwtKey123456789  
jwt.expiration=900000
```

User Model

```
public class AppUser {  
  
    private Long id;  
    private String username;  
    private String password;  
    private String role;  
    private boolean enabled;  
  
    // getters & setters  
}
```

JDBC Repository

@Repository

```
public class UserRepository {

    private final JdbcTemplate jdbcTemplate;

    public UserRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public Optional<AppUser> findByUsername(String username) {

        String sql = """
            SELECT USER_ID, USERNAME, PASSWORD, ROLE, ENABLED
            FROM USERS
            WHERE USERNAME = ?
        """;

        return jdbcTemplate.query(sql, rs -> {
            if (rs.next()) {
                AppUser user = new AppUser();
                user.setId(rs.getLong("USER_ID"));
                user.setUsername(rs.getString("USERNAME"));
                user.setPassword(rs.getString("PASSWORD"));
                user.setRole(rs.getString("ROLE"));
                user.setEnabled("Y".equals(rs.getString("ENABLED")));
                return Optional.of(user);
            }
            return Optional.empty();
        }, username);
    }
}
```

UserDetailsService (Spring Security Bridge)

```
@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository repository;

    public CustomUserDetailsService(UserRepository repository) {
        this.repository = repository;
    }

    @Override
    public UserDetails loadUserByUsername(String username) {

        AppUser user = repository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found"));

        return User.builder()
            .username(user.getUsername())
            .password(user.getPassword())
            .roles(user.getRole())
            .disabled(!user.isEnabled())
            .build();
    }
}
```

JWT Utility

@Component

```
public class JwtUtil {

    @Value("${jwt.secret}")
    private String secret;

    @Value("${jwt.expiration}")
    private long expiration;

    public String generateToken(String username) {
        return Jwts.builder()
            .setSubject(username)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + expiration))
            .signWith(Keys.hmacShaKeyFor(secret.getBytes()), SignatureAlgorithm.HS256)
            .compact();
    }

    public String extractUsername(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(secret.getBytes())
            .build()
            .parseClaimsJws(token)
            .getBody()
            .getSubject();
    }
}
```

JWT Authentication Filter

@Component

```
public class JwtAuthenticationFilter extends OncePerRequestFilter {  
  
    private final JwtUtil jwtUtil;  
    private final CustomUserDetailsService userDetailsService;  
  
    public JwtAuthenticationFilter(JwtUtil jwtUtil,  
                                   CustomUserDetailsService userDetailsService) {  
  
        this.jwtUtil = jwtUtil;  
        this.userDetailsService = userDetailsService;  
    }  
}
```

@Override

```
protected void doFilterInternal(  
    HttpServletRequest request,  
    HttpServletResponse response,  
    FilterChain filterChain) throws ServletException, IOException {  
    String header = request.getHeader("Authorization");  
    if (header != null && header.startsWith("Bearer ")) {  
        String token = header.substring(7);  
        String username = jwtUtil.extractUsername(token);  
  
        UserDetails userDetails =  
            userDetailsService.loadUserByUsername(username);  
  
        UsernamePasswordAuthenticationToken auth =  
            new UsernamePasswordAuthenticationToken(  
                userDetails, null, userDetails.getAuthorities());  
  
        SecurityContextHolder.getContext().setAuthentication(auth);  
    }  
    filterChain.doFilter(request, response);  
}
```

Security Configuration (MOST IMPORTANT)

```
@Configuration
@EnableMethodSecurity
public class SecurityConfig {
    private final JwtAuthenticationFilter jwtFilter;
    public SecurityConfig(JwtAuthenticationFilter jwtFilter) {
        this.jwtFilter = jwtFilter;
    }

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(sm ->
                sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/login").permitAll()
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    AuthenticationManager authenticationManager(
        AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }
}
```

Login Request / Response (Java records)

```
public record LoginRequest(String username, String password) {}
public record LoginResponse(String token) {}
```

Auth Controller (LOGIN API)

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthenticationManager authManager;
    private final JwtUtil jwtUtil;

    public AuthController(AuthenticationManager authManager,
                           JwtUtil jwtUtil) {
        this.authManager = authManager;
        this.jwtUtil = jwtUtil;
    }

    @PostMapping("/login")
    public LoginResponse login(@RequestBody LoginRequest request) {

        Authentication authentication =
            authManager.authenticate(
                new UsernamePasswordAuthenticationToken(
                    request.username(),
                    request.password()
                )
            );

        String token = jwtUtil.generateToken(authentication.getName());
        return new LoginResponse(token);
    }
}
```

Protected REST API Example

```
@RestController
@RequestMapping("/api/students")
public class StudentController {

    @GetMapping
    public String secureEndpoint() {
        return "You are authenticated!";
    }

    @PreAuthorize("hasRole('ADMIN')")
    @GetMapping("/admin")
    public String adminOnly() {
        return "Admin access only";
    }
}
```

How to Test (Postman)

Step 01: Login and get token

Login (get token)

```
POST /api/auth/login HTTP/1.1
Host: localhost:8080
Content-Type: application/json

{
  "username": "admin",
  "password": "password123"
}
```

Login Response

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhZG1pbj9..."
}
```

Step 02 : POST request with token (create incident)

```
curl -X POST http://localhost:8080/api/incidents \
-H "Authorization: Bearer eyJhbGciOiJIUzI1NiJ9..." \
-H "Content-Type: application/json" \
-d '{
  "title": "Server down",
  "priority": "HIGH"
}'
```

MEMORIZE THIS

Spring Security + JWT + BCrypt + Stateless authentication

***** Front end – Spring Boot (REST API) *****

Where do HTML pages live in Spring Boot?

Spring Boot serves **static content** from these folders (in this order):

```
src/main/resources/  
├─ static/  
├─ public/  
├─ resources/  
└─ META-INF/resources/
```

Recommended for simple HTML (Login page)

```
src/main/resources/static/login.html
```

Then the page is available at:

```
http://localhost:8080/login.html
```

Project Structure (Typical)

```
src/main/java  
└─ com.example.app  
    ├─ controller  
    ├─ service  
    └─ repository
```

```
src/main/resources  
├─ static  
│   ├─ login.html  
│   ├─ css/  
│   └─ js/  
└─ application.yml
```

Simple login.html (Frontend)

```
<!DOCTYPE html>
<html>
<head>
  <title>Login</title>
  <meta charset="UTF-8">
</head>
<body>

<h2>Login</h2>

<form id="loginForm">
  <label>Username:</label>
  <input type="text" id="username" required><br><br>

  <label>Password:</label>
  <input type="password" id="password" required><br><br>

  <button type="submit">Login</button>
</form>

<p id="result"></p>
<script>
  document.getElementById("loginForm").addEventListener("submit", function(e) {
    e.preventDefault();

    fetch('/api/login', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        username: document.getElementById("username").value,
        password: document.getElementById("password").value
      })
    })
    .then(res => res.json())
    .then(data => {
      document.getElementById("result").innerText = data.message;
    })
    .catch(err => console.error(err));
  });
</script>
</body>
</html>
```

REST Login API (Spring Boot)

Login Controller

```
@RestController
@RequestMapping("/api")
public class LoginController {

    private final LoginService loginService;

    public LoginController(LoginService loginService) {
        this.loginService = loginService;
    }

    @PostMapping("/login")
    public Map<String, String> login(@RequestBody LoginRequest request) {
        boolean success = loginService.authenticate(
            request.getUsername(),
            request.getPassword()
        );

        if (success) {
            return Map.of("message", "Login successful");
        }
        return Map.of("message", "Invalid credentials");
    }
}
```

LoginRequest DTO

```
public class LoginRequest {
    private String username;
    private String password;

    // getters & setters
}
```

Service Layer (Oracle DB Check)

```
@Service
public class LoginService {

    private final JdbcTemplate jdbcTemplate;

    public LoginService(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public boolean authenticate(String username, String password) {
        String sql = ""
            SELECT COUNT(*)
            FROM USERS
            WHERE USERNAME = ? AND PASSWORD = ?
        "";

        Integer count = jdbcTemplate.queryForObject(
            sql,
            Integer.class,
            username,
            password
        );

        return count != null && count > 0;
    }
}
```

Oracle USERS Table (Example)

```
CREATE TABLE USERS (
    ID NUMBER GENERATED BY DEFAULT AS IDENTITY,
    USERNAME VARCHAR2(50) UNIQUE,
    PASSWORD VARCHAR2(100),
    PRIMARY KEY (ID)
);
```

Other logic

Implement other logic that you need to authenticate

When to use templates/ instead of static/?

- You want **server-side rendering**
- You need dynamic HTML from backend

**** Handling session id ****

After authentication using REST, you usually do NOT manually store or resend the sessionId yourself. The browser stores it automatically as a cookie (JSESSIONID) and sends it with every request.

How Session Works in Spring (Under the Hood)

When authentication is successful:

```
HttpSession session = request.getSession(true);
session.setAttribute("USER", user);
```

Spring automatically:

- Creates a session
- Generates a session ID
- Sends it to the browser as a cookie

Response header sent by Spring:

```
Set-Cookie: JSESSIONID=ABC123XYZ; Path=/; HttpOnly
```

Where Is the Session ID Stored?

✓ In the Browser (Automatically)

The browser stores it in **cookies**.

You can see it in:

- Chrome → DevTools → Application → Cookies
- Firefox → Storage → Cookies

You **do not need to write code** to store it.

How Is Session ID Sent to Other REST APIs?

✓ Automatically by the browser

For **same domain requests**, the browser adds:

```
Cookie: JSESSIONID=ABC123XYZ
```

So your next REST call:

```
GET /api/incidents
```

Automatically includes the session cookie.

Example: Frontend Fetch Call (Correct Way)

If a valid `JSESSIONID` cookie already exists in the browser, you do NOT manually add it.
You must tell the browser to SEND cookies using `credentials`.

Pre-condition (Already done in your case)

- ✓ User logged in successfully
- ✓ Browser has this cookie:
- ✓ Cookie is visible in DevTools → Application → Cookies

HTML + JS REST Call (MOST IMPORTANT)

```
<script>
function loadIncidents() {
    fetch('/api/incidents', {
        method: 'GET',
        credentials: 'same-origin' // ● THIS IS THE KEY
    })
    .then(response => {
        if (response.status === 401) {
            alert("Session expired. Please login again.");
            return;
        }
        return response.json();
    })
    .then(data => console.log(data))
    .catch(err => console.error(err));
}
</script>

<button onclick="loadIncidents()">Get Incidents</button>
```

What happens internally?

- Browser finds `JSESSIONID` cookie
- Browser sends it automatically:

Cookie: `JSESSIONID=ABC123XYZ`

- Spring matches the session
- User is authenticated

Cross-Origin Case (Frontend & Backend on different ports/domains)

- Frontend: `http://localhost:3000`
- Backend: `http://localhost:8080`

✓ Frontend (MANDATORY)

```
fetch('http://localhost:8080/api/incidents', {  
  method: 'GET',  
  credentials: 'include' // 🚫 REQUIRED  
})
```

✓ Backend (MANDATORY)

```
@CrossOrigin(  
    origins = "http://localhost:3000",  
    allowCredentials = "true"  
)  
@RestController  
@RequestMapping("/api")  
public class IncidentController {  
}
```

✗ Without this → cookie will **not** be sent.

What NOT to Do

Manually adding sessionId, Adding headers manually, Returning sessionId in JSON and storing it

Backend: How Spring Reads the Session

```
@GetMapping("/api/incidents")  
public List<Incident> getIncidents(HttpSession session) {  
  
    Object user = session.getAttribute("USER");  
  
    if (user == null) {  
        throw new RuntimeException(HttpStatus.UNAUTHORIZED);  
    }  
  
    return incidentService.getIncidents();  
}
```

✓ Same session ✓ Same user ✓ No token handling